

ATEK RF Modules

Programming Manual

USB remote control: SCPI command set and Python API

Document revision 1.0 | Python API version 1.0.0

ATEK MIDAS

Contents

1 Introduction.....	3
1.1 Documentation structure.....	3
1.2 Software download.....	3
1.3 Conventions.....	3
2 Connection.....	3
3 Command Reference.....	4
3.1 Syntax.....	4
3.2 Commands.....	4
3.3 States.....	4
3.4 State reporting.....	4
3.5 Typical sequence.....	5
4 Raw Command Example.....	5
5 Python API.....	5
5.1 Overview.....	5
5.2 Requirements and installation.....	5
5.3 Quick start.....	6
5.4 Connecting.....	6
5.5 Methods common to all modules.....	7
5.6 Module-specific classes.....	7
5.7 Confirmation (verify).....	7
5.8 Callbacks and threading.....	7
5.9 Errors.....	8
5.10 Example program.....	8
6 Firmware Update (USB DFU).....	8
7 Troubleshooting.....	9

1 Introduction

ATEK USB-controlled RF modules can be operated from their front-panel buttons or remotely from a computer. This manual describes remote control. Two methods are available:

- **Raw commands:** a short SCPI-like text protocol over a USB virtual serial port. It can be used from any programming language or serial terminal.
- **Python API:** the file `atek_rf_modules_scpi_api.py`, which wraps the command set in a thread-safe, object-oriented interface.

1.1 Documentation structure

Remote control is documented in two levels:

- **This manual** describes everything that is common to all modules: connection, command set, state reporting, Python API, firmware update and troubleshooting.
- **A Programming Supplement for each module** gives the module-specific information: identification, the meaning of each state, raw command examples and the module's Python class with an example program.

Available modules and supplements:

Module	RF chip	Description	Document
ATEK1801	ATEK888P5	20-550 MHz 32-State USB-Controlled Low Pass Filter	ATEK1801 Programming Supplement
ATEK1601	ATEK656N5	2-18 GHz Sub-Octave USB-Controlled Filter Bank	ATEK1601 Programming Supplement

The **Module** name is the identifier returned by the `*IDN?` command. The **RF chip** column gives the RF device used inside the module; RF performance data is found in the datasheet of that chip.

1.2 Software download

The Python API, an example program, the ATEK user interface and the module datasheets are published in the ATEK MIDAS GitHub repository:

<https://github.com/atek-midas/atek-rf-modules>

The repository README describes where each file is located. Always use the latest version from the repository.

1.3 Conventions

Short code examples are included in this document. Complete example programs are not printed; they are available in the repository (Section 1.2).

2 Connection

The module is a USB CDC (Communication Device Class) device and appears as a virtual serial port. Windows 10/11, Linux and macOS include the required driver; no installation is necessary.

Operating system	Port name
Windows	COMx (see Device Manager → Ports)
Linux	<code>/dev/ttyACMx</code>
macOS	<code>/dev/cu.usbmodemXXXX</code>

- Baud rate, data bits, parity and stop bits have no effect on a USB CDC link. Any setting can be used (for example 115200, 8N1).

- A serial port can be opened by only one program at a time. Close the ATEK user interface or any terminal before running your own software.
- On Linux the user must have access to the port, usually by membership of the dialout group.

3 Command Reference

3.1 Syntax

- Commands are ASCII text, terminated by CR, LF or CR+LF.
- Commands are case-sensitive and must be written as shown (upper case).
- One command per line, maximum 63 characters.
- Responses are terminated by CR+LF.
- Unknown commands are ignored and produce no response.

3.2 Commands

Command	Type	Description	Response
*IDN?	Query	Returns the module name.	e.g. ATEK1601
GET?	Query	Returns the current state index.	STATE:<n>
SET:<n>	Set	Selects state n (decimal integer).	STATE:<n> if accepted, nothing if n is out of range
SET:UPDATE	Action	Starts the USB firmware update mode (Section 6).	Entering DFU Mode..., then USB disconnects

3.3 States

Every module has a fixed number of states, numbered from 0. The state index is used by SET:<n> and reported by STATE:<n>. The meaning of each state (attenuation, filter band, ...) and the valid range are given in the Programming Supplement of the module. The front-panel buttons step through the same states.

3.4 State reporting

The module sends STATE:<n> whenever its state changes, regardless of the source of the change:

- after an accepted SET:<n> command (also when n equals the current state),
- after a change made with the front-panel buttons,
- as the response to GET?.

This keeps any connected software synchronised with the module, including when the module is operated by hand. Applications should therefore observe the following rules:

- A STATE:<n> line can arrive at any time. When waiting for the response to *IDN?, skip STATE: lines.
- To confirm a SET:<n>, wait for the matching STATE:<n> (300 ms is ample). If it does not arrive, read the state with GET?.
- An out-of-range SET:<n> is ignored without a response. Check the range before sending.

WARNING The argument of SET: is read as a decimal number. A non-numeric argument (for example SET:ABC) is interpreted as 0 and selects state 0.

3.5 Typical sequence

Direction	Line	Comment
PC → Module	*IDN?	Identify the module
Module → PC	ATEK1601	
PC → Module	GET?	Read the current state
Module → PC	STATE:0	
PC → Module	SET:3	Select state 3
Module → PC	STATE:3	Confirmation
Module → PC	STATE:4	Front-panel button pressed

4 Raw Command Example

The commands can be sent from any language that can open a serial port. The following Python example uses only the `pyserial` package, without the ATEK API:

```
import serial

with serial.Serial("COM5", 115200, timeout=1) as port:    # "/dev/ttyACM0" on Linux
    port.write(b"*IDN?\r\n")
    print(port.readline().decode().strip())    # module name, e.g. ATEK1601

    port.write(b"SET:3\r\n")
    print(port.readline().decode().strip())    # STATE:3

    port.write(b"GET?\r\n")
    print(port.readline().decode().strip())    # STATE:3
```

This minimal example assumes that no front-panel button is pressed during the exchange. Robust software must handle unsolicited `STATE:` lines as described in Section 3.4; the Python API does this automatically.

For manual tests a serial terminal (for example PuTTY or Tera Term) can be used. Configure it to send CR+LF at the end of each line.

5 Python API

5.1 Overview

`atek_rf_modules_scpi_api.py` is a single Python file that implements the command set of Section 3. It provides:

- automatic module identification and a class for each module type,
- range checking before a command is sent and confirmation after it has been applied,
- a background reader thread that keeps track of state changes, including changes made with the front-panel buttons,
- thread-safe access, so one module object can be shared between threads,
- clear exceptions for every error condition.

5.2 Requirements and installation

- Python 3.8 or newer
- `pyserial`: `pip install pyserial`

Download `atek_rf_modules_scpi_api.py` from the repository (Section 1.2). No further installation is needed: copy the file into the folder of your script (or any folder on the Python path) and import it:

```
import atek_rf_modules_scpi_api as atek

print(atek.__version__)
```

5.3 Quick start

```
import atek_rf_modules_scpi_api as atek

with atek.connect("COM5") as dev:          # opens, identifies, returns module class
    print(dev.model)                       # module name, e.g. "ATEK1601"
    dev.set_state(3)                       # select state 3, waits for confirmation
    print(dev.get_state())                 # 3
# port is closed automatically
```

5.4 Connecting

Function	Description
<code>connect(port, **options)</code>	Opens the port, identifies the module and returns an object of the matching class (Section 5.6). Raises <code>AtekConnectionError</code> if the port cannot be opened and <code>AtekTimeoutError</code> if no ATEK module answers.
<code>find_devices(ports=None, timeout=0.5)</code>	Probes serial ports with <code>*IDN?</code> and returns a list of <code>DeviceInfo(port, model, description)</code> . Scans all ports when <code>ports</code> is not given.
<code>list_serial_ports()</code>	Returns the names of all serial ports on the computer.

Options of `connect()`: `timeout` (response timeout in seconds, default 1.0) and the callbacks described in Section 5.8.

NOTE `find_devices()` sends `*IDN?` to every probed port. If other serial instruments are connected, pass a list of ports to limit the scan.

Close the connection with `dev.close()`, or use a `with` block as shown above.

5.5 Methods common to all modules

Method / property	Description
<code>set_state(state, verify=True)</code>	Selects a state index. Returns the applied state.
<code>get_state()</code>	Reads the current state index from the module (GET?).
<code>identify()</code>	Sends *IDN?, returns the module name.
<code>model</code>	Module name, e.g. "ATEK1601".
<code>info</code>	ModuleInfo with model, chip, description, num_states, max_state.
<code>num_states</code>	Number of states of the module.
<code>last_state</code>	Last state reported by the module. Updated in the background; no communication.
<code>is_open</code>	True while the connection is open.
<code>write(command)</code>	Sends a raw command line.
<code>query(command, timeout=None)</code>	Sends a raw command and returns the first response line.
<code>enter_dfu()</code>	Starts firmware update mode (Section 6) and closes the connection.
<code>close()</code>	Closes the connection.

5.6 Module-specific classes

`connect()` returns an object of the class that matches the connected module. The class adds methods that work in physical units (band, MHz, ...); the common methods above remain available. The methods of each class are described in the Programming Supplement of the module.

Module	Python class	Document
ATEK1801	<code>atek.TunableLowPassFilter</code>	ATEK1801 Programming Supplement
ATEK1601	<code>atek.ATEK1601</code>	ATEK1601 Programming Supplement

5.7 Confirmation (verify)

All set methods accept a `verify` argument:

- `verify=True` (default): the method returns only after the module has confirmed the new state. If the confirmation is lost, the state is read back with GET?. `AtekStateError` is raised if the module did not apply the state. Use this for automated measurements.
- `verify=False`: the command is sent and the method returns immediately. Use this for fast sweeps or graphical controls such as sliders; confirm later with `get_state()` if needed.

5.8 Callbacks and threading

Callbacks are optional. Pass them to `connect()` or assign them to the object at any time:

Callback	Called when
<code>on_state_change(state)</code>	a STATE:<n> message is received: after SET, after GET? and after a front-panel button press
<code>on_tx(line)</code>	a line is sent to the module
<code>on_rx(line)</code>	a line is received from the module
<code>on_disconnect(error)</code>	the connection is lost unexpectedly (e.g. cable removed)

```
def state_changed(state):
    print("Module is now in state", state)

dev = atek.connect("COM5", on_state_change=state_changed)
```

Callbacks run in the background reader thread. Keep them short and do not call them from a GUI toolkit directly: Tkinter, Qt and similar libraries must only be updated from their main thread. Pass the value to the main thread, for example through a queue .Queue. Exceptions raised inside a callback are ignored so that communication continues.

5.9 Errors

Exception	Raised when
AtekError	Base class of all exceptions below. Also raised for an unknown module ID.
AtekConnectionError	the port cannot be opened, or the connection is lost
AtekTimeoutError	the module does not answer within the timeout
AtekStateError	an invalid state, band or value is requested, or the module did not apply it

5.10 Example program

The example program `example_basic_usage.py` is available in the repository (Section 1.2). It works with every module: it finds the module (or uses the port given on the command line), reads the module identification, selects a state and reads the state back.

Run it without arguments to find the module automatically, or give the port name, for example `python example_basic_usage.py COM5`. Example output:

```
Port   : COM5
IDN    : ATEK1601
State  : 1
```

Examples that use the module-specific classes are given in the Programming Supplement of each module.

6 Firmware Update (USB DFU)

The command `SET:UPDATE` places the module in the factory ROM bootloader of its STM32 microcontroller (USB DFU mode). New firmware can then be loaded over the same USB cable, without opening the enclosure or shorting the BOOT0 pin on the board.

1. Close all software that uses the module port.
2. Send `SET:UPDATE`, or call `dev.enter_dfu()` in Python. The module answers `Entering DFU Mode...` and shows `BOOT MODE` on its display.
3. The virtual serial port disappears. The module re-enumerates as a USB DFU device (STM32 BOOTLOADER, USB ID 0483:DF11).
4. Load the firmware file with STM32CubeProgrammer (USB interface) or `dfu-util`. On Windows the STM32 DFU driver is installed together with STM32CubeProgrammer.
5. Disconnect and reconnect the USB cable. The module starts with the new firmware and the virtual serial port is available again.

WARNING RF control is not available while the module is in DFU mode. Use only firmware files provided by ATEK MIDAS and do not disconnect the module while programming is in progress. If programming is interrupted, contact ATEK MIDAS support.

7 Troubleshooting

Symptom	Remedy
No serial port appears	Use a USB data cable (not a charge-only cable). Check Device Manager (Windows) or dmesg (Linux).
Port cannot be opened / access denied	Close other programs using the port (ATEK user interface, terminals). On Linux, add the user to the dialout group.
No response to commands	Terminate every command with CR, LF or CR+LF. Commands are case-sensitive.
SET has no effect	The state is out of range for this module; see the Programming Supplement of the module.
Unexpected STATE: messages	Normal behaviour: the front-panel buttons were used (Section 3.4).
AtekTimeoutError on connect	Wrong port selected or module not connected. Use <code>find_devices()</code> to locate the module.